BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **MICHAL REPČÍK**
AUTOR PRÁCE

SUPERVISOR **Ing. KAREL FRITZ,**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

REPČÍK, Michal. *3D Neural Cellular Automata*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT to correct spelling and other language mistakes. I used Grok when working on the software.

. . . . . .
Michal Repčík
November 30, 2025

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1 Introduction **3**

2 Abstract **4**

3 Theoretical Foundations **5**

3.1 Prerequisites 5

3.2 Cellular Automata 5

3.3 Neural Cellular Automata 7

3.4 Core Components of a 3D Neural Cellular Automaton 8

4 Releated Work **11**

5 Implementation **12**

6 Conclusion **13**

Bibliography **14**

List of Figures

Chapter 1

Introduction

TODO

Chapter 2

Abstract

TODO

Chapter 3

Theoretical Foundations

3.1 Prerequisites

This thesis assumes familiarity with fundamental concepts in modern deep learning, in particular convolutional neural networks (CNNs), gradient-based optimisation, and automatic differentiation as implemented in frameworks such as PyTorch [18, 5]. Comprehensive introductions to these topics are provided in Goodfellow et al. [8] and in the official PyTorch documentation [5].

The remainder of the thesis is self-contained with respect to cellular automata (CA) and their neural extensions.

3.2 Cellular Automata

Cellular automata (CA) are discrete dynamical systems composed of a regular grid of cells, each of which occupies one state from a finite set. The grid evolves in parallel over discrete time steps according to a local update rule that depends solely on the states of neighbouring cells [13].

Formally, a cellular automaton is defined by:

- a **lattice** of cells, typically arranged in one, two, or three dimensions,
- a **finite state set** $\mathcal{S}$,
- a **neighbourhood function** $\mathcal{N}$ describing which cells influence a given cell,
- an **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \to \mathcal{S}$ that determines the next state [10].

The foundational work of Ulam and von Neumann in the 1940s established CA as a model for self-replication [16]. Later, Wolfram proposed a widely used behavioural classification of CA into four qualitative classes:

- **Class 1 - Uniform**: quickly reaches a single state.
- **Class 2 - Periodic**: static or oscillating patterns.
- **Class 3 - Chaotic**: unpredictable, noisy evolution.
- **Class 4 - Complex**: long-lived structures and computation [24, 25].

These classes remain a reference point in subsequent work on both classical and neural CA.

3.2.1 One-Dimensional Cellular Automata

The simplest non-trivial CA operate on a one-dimensional array of cells. In the elementary CA family, each cell observes its own state and that of its two immediate neighbours (radius 1, neighbourhood size 3). With binary states, the update rule has $2^3 = 8$ possible neighbourhood configurations, yielding $2^8 = 256$ distinct rules in Wolfram's enumeration scheme [24].

A rule number encodes the next-state lookup table in binary. Notable examples include:

- **Rule 90**, which generates the well-known Sierpiński triangle from a single active seed (Class 3),
- **Rule 30**, whose chaotic evolution has been used as a source of pseudorandomness,
- **Rule 110**, (Class 4) is the smallest known Turing-complete 1D CA, supporting gliders and glider collisions that can perform any computation [6].

3.2.2 Two-Dimensional Cellular Automata

In 2D, the Moore neighbourhood (a 3×3 block) is the most common. Because the space of lookup tables is large (2^9 input combinations), rules are often expressed using birth/survival notation.

Conway's Game of Life [7] remains the most influential 2D CA, featuring:

- still lifes (e.g. block, beehive),
- oscillators (e.g. blinker, toad),
- spaceships (e.g. glider, lightweight spaceship),
- glider guns and other computational structures.

Life is Turing-complete [1] and has influenced much of the later work on self-organising neural CA models.

3.2.3 Three-Dimensional Cellular Automata

In three dimensions, the Moore neighbourhood includes up to 26 adjacent cells (face-, edge-, and corner-adjacent) in a $3\times3\times3$ cube. With binary states, this creates $2^{27} \approx 134$ million possible neighbourhood configurations, so practical 3D CA typically use totalistic or outer-totalistic rules that depend only on the sum of live neighbours (not their exact positions) [25].

Notable examples include:

- von Neumann's 29-state self-replicating automaton [16],
- excitable-media rules such as Brian's Brain extended to 3D [12],
- 3D Game of Life variants producing branching and crystal-like growths.

Advances in computing hardware in the 2000s facilitated larger-scale 3D CA studies [23]. These volumetric models later aligned naturally with neural CA research.

3.2.4 Differentiability and Continuous Cellular Automata

Although this thesis uses a discrete 3D lattice, the update rule is fully differentiable, enabling gradient-based optimisation [2]. This connects neural CA to continuous cellular automata, where states are real-valued and update rules are smooth rather than discrete.

SmoothLife [19] generalises the Game of Life to continuous space and time using continuous averaging kernels and smooth transition functions, producing gliders and complex wave-like structures on a continuous field.

Lenia [3] further extends this idea using parameterised convolution kernels and growth functions. Its exploration of parameter space led to hundreds of stable, self-organising patterns resembling an artificial ecology.

These continuous systems demonstrate that local, differentiable interactions can support rich self-organisation, a property later leveraged by neural CA [15].

3.3 Neural Cellular Automata

Neural Cellular Automata (NCA) replace hand-crafted rules with a shared learnable update function parametrized by neural network. The grid remains discrete, but the cell state becomes a real-valued vector, making the entire system differentiable and trainable with gradient descent.

The foundational work in this area is Growing Neural Cellular Automata by Mordvintsev et al. [15]. The authors demonstrated that a small neural network operating locally at each cell can give rise to complex global behaviour. Their experiments reveal three characteristic training regimes:

- **Growing models**, trained solely to produce a target pattern from a small seed. The authors report that such models learn to generate patterns but not necessarily to preserve them, noting that some patterns may "explode," "decay," or become "almost stable" [15, Experiment 1].

- **Persistent models**, trained to maintain the generated pattern over hundreds of simulation steps. These models often develop limited regenerative abilities "without being explicitly instructed to do so" [15, Experiment 2].

- **Regenerating models**, trained under random damage—large regions of the pattern are zeroed at each step. As observed in the original experiments, such models acquire strong regenerative capabilities, "especially in the central area" [15, Experiment 3].

Across all regimes, the architecture consists of a small update network (approximately 100 parameters) and a fixed set of local perception filters (identity, Sobel-like gradients, and blur kernels). The training objective-growth, persistence, or regeneration—is the key distinguishing factor.

Subsequent work explored memory channels [20], self-classifying multicellular structures [9], and other pattern-forming systems. While most research on neural cellular automata remains focused on two-dimensional grids, several early voxel-based 3D extensions have been proposed [21, 22, 11], demonstrating that the principles of learned local update rules can be scaled to volumetric domains.

3.4 Core Components of a 3D Neural Cellular Automaton

The previous sections introduced classical cellular automata, continuous extensions, and existing neural CA models. The following section focuses on the main theoretical components that form the basis of the three-dimensional neural cellular automaton used later in the thesis. The goal is to clarify the general concepts without referring to specific implementation details.

3.4.1 State Representation

In neural cellular automata, each voxel holds a vector of real-valued channels. This concept originates from the 2D model of Mordvintsev et al. [15], where multi-channel states support both visible pattern information and internal memory.

The channels typically fall into two categories:

- **Visible channels**, which represent quantities compared directly to the target pattern (for example colour, density, or occupancy),
- **Hidden channels**, which provide additional internal memory and facilitate communication across neighbours.

If each cell has C channels, its state can be written as

$$\mathbf{s} \in \mathbb{R}^C.$$

Many neural CA models also allocate a continuous *alive* channel, which acts as a gating mechanism controlling whether updates are applied to the cell. This mechanism appears in earlier NCA work [15] and has conceptual links to continuous CA such as SmoothLife [19] and Lenia [3], where real-valued states enable smooth evolution and gradient-based optimisation.

3.4.2 Perception Mechanism

Despite using continuous state vectors, neural CA retain the defining CA property of locality: each cell updates based solely on information from its neighbourhood [10, 25]. In neural CA, locality is achieved by applying a fixed set of convolutional filters (e.g., 3×3 in 2D or 3×3×3 in 3D) to extract local spatial features [15, 22].

Common filter types include:

- **Identity filters**, capturing a cell's own state,
- **Neighbour aggregation filters**, such as sums or averages of face-adjacent neighbours,
- **Gradient filters**, including Sobel-type kernels extended to 3D, which approximate local directional differences and parallel biological gradient sensing [15, 22].

The outputs of all applied filters are concatenated into a single perception vector for each cell, which is passed to the update function. Using fixed (non-learnable) filters helps stabilise training and preserves the conceptual continuity between neural and classical cellular automata [15, 3].

3.4.3 Update Rule

The update rule in neural cellular automata is a shared, parametric function that determines how each cell's state evolves over time. Following the formulation introduced by Mordvintsev et al. [15], the rule operates on the perceived neighbourhood features and produces a state increment that is added to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

This formulation preserves the parallel and local nature of classical cellular automata while allowing the transition function to be learned through gradient-based optimisation.

The update function is typically realized as a lightweight neural network applied independently to every cell, such as a small multilayer perceptron or a sequence of 1×1 (or $1 \times 1 \times 1$ in 3D) convolutions [15]. These architectures ensure that the update rule depends only on locally perceived information while keeping the parameter count low enough to maintain stable training.

Two mechanisms commonly appear across neural CA formulations:

- **Alive gating**: a continuous "alive" signal modulates whether a cell applies the update, providing a soft analogue of classical alive/dead states while improving long-term stability [15].
- **Stochastic update masking**: a probabilistic update rate (or "fire rate") introduces randomness into the execution of the update rule, which helps prevent rigid synchronisation and encourages robustness [15].

These design elements support the emergence of behaviours such as growth, pattern persistence, and regeneration, phenomena also studied in continuous morphogenetic systems like Lenia [3]. Recent three-dimensional neural CA models [21, 22, 11] extend the same principles to volumetric grids, demonstrating that the underlying update-rule formulation generalises naturally beyond 2D.

3.4.4 Grid Management

The grid provides the spatial substrate on which the neural CA evolves. It is typically a finite 3D lattice, although different boundary conditions may be used depending on the experiment [15].

Important considerations include:

- **Dimensions and boundary conditions**: Fixed lattice sizes (e.g., 32^3) with periodic or zero-padding boundaries to avoid artefacts at edges [17].
- **Initialization**: Grids are commonly seeded with one or a few active voxels, sometimes with added noise to increase robustness [15].
- **Simulation loop**: The perception and update steps are iterated for multiple time steps; for training efficiency, simulations are often batched or pooled [3].

Grid representations in three dimensions are computationally demanding due to the cubic scaling of voxel-based environments. Sparse and block-based grid structures are commonly used in related 3D modelling and simulation domains [14, 4], and represent potential strategies for scaling neural cellular automata to larger volumes.

3.4.5 Training and Optimisation

Because neural CA are differentiable, they can be optimised using gradient-based methodsto minimise a loss between the generated pattern and a target configuration [15]. Training typically involves unrolling the CA dynamics over a number of steps and applying back-propagation through time.

Major components include:

- **Loss functions**: Mean squared error (MSE) is commonly used for continuous targets, while structural tasks may use intersection-over-union or other domain-specific metrics [15, 17].
- **Training regimes**: Growth, persistence, and regeneration training regimes induce distinct behaviours and robustness properties [15].
- **Optimisation strategies**: Adam is widely used with learning rates around 0.001, often supplemented by early stopping, data pooling, or logging frameworks [22].

These training strategies allow the discovery of update rules capable of producing stable and robust self-organising structures in both 2D and early 3D systems [21, 22, 11].

Chapter 4

Releated Work

TODO

Chapter 5

Implementation

TODO

Chapter 6

Conclusion

TODO

Bibliography

[1] Berlekamp, E. R.; Conway, J. H. and Guy, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.

[2] Béna, G.; Faldor, M.; Goodman, D. F. M. and Cully, A. *A Path to Univrsal Neural Cellular Automata*. 2025. Available at: https://arxiv.org/abs/2505.13058.

[3] Chan, B. W.-C. *Lenia: Biology of Artificial Life*. 2019.

[4] Choy, C.; Gwak, J. and Savarese, S. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[5] Community, P. *PyTorch Documentation*. 2025. Available at: https://docs.pytorch.org/stable/index.html.

[6] Cook, M. *Universality in Elementary Cellular Automata*. 2004.

[7] Gardner, M. *Mathematical Games: The fantastic combinations of John Conway's new solitaire game "Life"*. 1970. Available at: https://web.stanford.edu/class/sts145/Library/life.pdf.

[8] Goodfellow, I.; Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[9] Horváth, D. and Levin, M. *Self-Classifying MNCA Distill*. 2021. Available at: https://distill.pub/2021/self-classifying-mnca/.

[10] Ilachinski, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001. ISBN 981-02-4623-4.

[11] Jha, A. *3D Growing Neural Cellular Automata*. GitHub, 2024. Available at: https://github.com/Aadityaza/3d-Growing-neural-cellular-automata. Originally published under username Aadityaza.

[12] Kinderen, E. de and Wuensche, A. 3D CA in a fluid dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems: From Biology to Hardware*. Springer, 1998.

[13] Marinescu, D. C. Chapter 2 - Nature-Inspired Algorithms and Systems. In: Marinescu, D. C., ed. *Complex Systems and Clouds*. Boston: Elsevier, 2017, p. 33–63. Computer Science Reviews and Trends. ISBN 978-0-12-804041-6. Available at: https://www.sciencedirect.com/science/article/pii/B9780128040416000025.

[14] Mildenhall, B.; Srinivasan, P. P.; Tancik, M.; Barron, J. T.; Ramamoorthi, R. et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2020.

[15] Mordvintsev, A.; Randazzo, E.; Niklasson, E. and Levin, M. *Growing Neural Cellular Automata*. 2020. Available at: https://distill.pub/2020/growing-ca/.

[16] Neumann, J. von. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.

[17] Niklasson, E.; Mordvintsev, A.; Randazzo, E. and Levin, M. Self-Organising Textures. *Distill*, 2021. Https://distill.pub/selforg/2021/textures/.

[18] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available at: https://arxiv.org/abs/1912.01703.

[19] Rafler, S. *Generalization of Conway's "Game of Life" to a continuous domain – SmoothLife* arXiv preprint arXiv:1111.1567. 2011. Available at: https://doi.org/10.48550/arXiv.1111.1567.

[20] Randazzo, E.; Mordvintsev, A. and Levin, M. *Persistent Memory in Growing Neural Cellular Automata* Google Research Blog. 2021. Available at: https://ai.googleblog.com/2021/12/persistent-memory-in-neural-cellular.html.

[21] Rossi, F. and Orsini, M. *3D Growing Neural Cellular Automaton*. GitHub, 2021. Available at: https://github.com/SkyLionx/3d-cellular-automaton. Originally published under username SkyLionx.

[22] Sudhakaran, S.; Grbic, D.; Li, S.; Katona, A.; Najarro, E. et al. *Growing 3D Artefacts and Functional Machines with Neural Cellular Automata*. July 2021, ALIFE 2021: The 2021 Conference on Artificial Life, p. 108. ALIFE 2022: The 2022 Conference on Artificial Life. Available at: https://doi.org/10.1162/isal_a_00451.

[23] Tobias, P. A. G. *3D Cellular Automata*. 2009. Available at: https://eprints.bournemouth.ac.uk/17408/.

[24] Wolfram, S. *Statistical Mechanics of Cellular Automata*. 1983.

[25] Wolfram, S. *A New Kind of Science*. Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: https://www.wolframscience.com/nks/.